

ChaosProbe thesis strategy review

Bottom line

I was able to inspect the public `yhu02/uva-msc-thesis` repository and its `ChaosProbe` documentation. The public repo already exposes a real experiment system rather than a slideware prototype: a Python CLI, a placement-strategy matrix, multi-fault execution, continuous metrics collection, Neo4j export, route-level latency joins, bootstrap confidence intervals, pairwise tests, and a `doctor` command that checks artifact quality and provenance. That is enough substrate for a serious MSc thesis if you turn it into a **careful measurement study** instead of a universal scheduler claim. ¹

The strongest defensible thesis is **not** “I found the best Kubernetes placement strategy” and **not** “I refuted the placement literature.” The strongest defensible thesis is: **placement effects under chaos are fault-class-specific and metric-layer-specific; under single-replica pod-delete churn, placement can move kernel/network reconvergence signals without producing a stable user-visible effect, while under latency- or contention-dominated faults, placement is more likely to matter at the user-visible layer.** That framing fits your current tool, lines up with the strongest signal already present in the repo, and avoids claims your current evidence cannot safely carry. ²

Bluntly: **yes, this can become a worthy MSc systems thesis.** But only if you narrow the contribution to a bounded empirical claim, clean the evidence chain, stop saying “refuted” where you mean “inapplicable in this regime,” and choose one strong positive or negative result as the centerpiece rather than trying to win every debate at once. The repo’s own docs already show why: the aggregate score is noisy, the contrack signal is strong, and the load-based locality result is explicitly marked preliminary because of dirty provenance and low replication. ³

The best primary research question is:

Under which chaos fault classes does pod placement measurably affect mechanism-level behavior and user-visible outcomes in a Kubernetes microservice application, and when do aggregate resilience scores obscure those effects?

That question is stronger than your current wording because it converts a thesis from “placement ranking” into a **fault-class-by-measurement-layer study**, which matches both the tool’s architecture and the literature gap. Existing Kubernetes docs and scheduling papers talk about availability, spreading, locality, and resource-fit, but they do not directly answer the question your tool can answer: **when a placement effect appears under chaos, at what layer does it appear, and does it propagate to the user?** ⁴

The strongest title is:

Option	Verdict
When Pod Placement Matters Under Chaos	Best balance of ambition and defensibility
Fault-Class-Specific Effects of Kubernetes Pod Placement Under Chaos	Most thesis-like and precise
Mechanism, Outcome, and Aggregate Score Under Kubernetes Chaos	Strong if you lean into layered measurement

The best single-sentence contribution statement is:

This thesis presents ChaosProbe, a Kubernetes chaos-evaluation framework, and uses it to show that pod placement effects are fault-class-specific and measurement-layer-specific: under single-replica churn, placement shifts reconvergence mechanisms more than user-visible outcomes, while aggregate resilience scores can fail to rank strategies reliably. ⁵

Literature fit

Your thesis sits at the intersection of **Kubernetes scheduling, microservice locality and tail latency, Kubernetes network reconvergence under endpoint churn, and chaos-engineering measurement frameworks**. That is a legitimate systems niche. It is stronger than a one-off chaos case study because it connects a cluster-control decision variable, multiple fault classes, and layered measurements. ⁶

The first literature stream is scheduler behavior and placement policy. Official Kubernetes docs describe a scheduler that filters feasible nodes and then scores them according to active rules; topology spread constraints exist specifically to spread pods across failure domains for higher availability, and `NodeResourcesFit` supports both spreading and bin-packing style scoring. That means two things. First, your “default” and “best-fit” strategies are well grounded in real scheduler behavior. Second, any claim that Kubernetes “only” optimizes resource fit and not availability is too blunt, because upstream Kubernetes explicitly supports topology spreading, inter-pod anti-affinity, and disruption budgets for availability-sensitive workloads. ⁷

The classic scheduler papers give you the comparison set, but not your answer. Borg is about large-scale cluster management and efficient task packing; Medea adds explicit placement constraints for long-running applications; Sparrow explores low-latency randomized scheduling; Quasar is interference- and QoS-aware rather than chaos-aware. These papers justify why placement policy matters, but they are largely about utilization, latency, or interference in shared clusters, not about **fault-class-specific chaos outcomes in Kubernetes microservices**. That gap is where your thesis should live. ⁸

A more direct adjacent stream is communication-aware and latency-aware Kubernetes scheduling. Recent work proposes scheduler extensions that account for microservice communication requirements, traffic history, or node-to-node latency because default scheduling does not explicitly model application communication structure. This literature supports your `dependency-aware` and locality-oriented ideas,

but again, it mostly focuses on QoS or edge latency rather than chaos fault classes. That makes it a **related but not duplicative** backdrop. ⁹

The benchmark and performance stream fits well. Online Boutique is a real, multi-service cloud-native microservice application used by Google to demonstrate Kubernetes, service-mesh, and gRPC-based systems, and its `cartservice` explicitly uses Redis. DeathStarBench exists because microservices change assumptions across the cloud stack and intensify tail-at-scale effects; Dean and Barroso’s “The Tail at Scale” remains the canonical reason to care about p95 and p99 rather than just mean latency. This gives you a clean motivation for emphasizing **user-visible route tails** over single aggregate scores. ¹⁰

The chaos-engineering stream also supports your direction, if you position it correctly. MicroRes proposes a resilience index via degradation dissemination from system metrics to user-aware metrics; ChaosEater automates the CE cycle with LLMs; Cast automates resilience testing in production cloud services; and the 2025 “Chaos Engineering in the Wild” study found that chaos usage in GitHub heavily emphasizes network disruption and instance termination while application-level faults are underrepresented. Your thesis should say: **ChaosProbe is not the first chaos platform, not the first resilience index, and not the first CE automation system. Its distinctive value is the controlled study of placement as the independent variable across fault classes with layered mechanism and user-visible measurements.** ¹¹

The network-reconvergence literature is where your current strongest result lives. Kubernetes docs state that EndpointSlices are the source of truth for kube-proxy’s internal routing decisions; kube-proxy runs on each node and reflects Service definitions into data-plane behavior. Upstream issues and blog posts document network programming latency concerns, intermittent connection resets, and conntrack cleanup/pathology when service endpoints change. That makes your current pod-delete/conntrack story plausible and well grounded. It is not hand-waving. It is an upstream-observed systems phenomenon. ¹²

That same literature also tells you where to be cautious. Endpoint handling and conntrack cleanup are evolving, and kube-proxy behavior is version-sensitive; for example, newer Kubernetes issues document conntrack reconciliation and cleanup regressions in later versions under heavy DNS churn. Since your documented cluster is Kubernetes `v1.28.6` on Calico with four 4 GiB workers, your findings are absolutely publishable as a bounded measurement study, but not as an “all Kubernetes behaves like this” claim. ¹³

A concise literature map looks like this:

Stream	What the literature says	What ChaosProbe can add
Kubernetes scheduling	Scheduler scores feasible nodes; supports topology spread, anti-affinity, and configurable bin packing. ¹⁴	Whether those placement choices matter differently under different chaos faults
Cluster schedulers	Borg, Medea, Sparrow, and Quasar justify packing, spreading, and QoS-aware placement. ⁸	A Kubernetes chaos measurement study rather than a new scheduler
Communication-aware placement	Recent work uses communication graphs and latency telemetry for better QoS. ⁹	Whether locality effects remain visible under failure injection

Stream	What the literature says	What ChaosProbe can add
Microservice benchmarks	Online Boutique and DeathStarBench are suitable microservice workloads; tails matter. ¹⁰	Route-specific, user-visible outcome measurement under controlled placement
Chaos frameworks	MicroRes, ChaosEater, Cast, and other CE work focus on profiling, automation, and production testing. ¹⁵	Placement as the manipulated variable with cross-layer metrics
Kubernetes churn mechanisms	EndpointSlice, kube-proxy, and conntrack can produce disruption windows under endpoint change. ¹⁶	Whether placement changes that mechanism and whether the user sees it

The best experiments

The experiments that matter are the ones that either produce a **clean negative result** or a **clean positive result**. Your current pod-delete story is a clean negative result at the user layer but a clean positive result at the mechanism layer. That is good science. Do not underrate it. A negative result with the right instrumentation is more defensible than a flashy but dirty claim. ¹⁷

The fastest path is to stop treating all strategies and all faults as equally important. A small number of carefully chosen strategies and fault classes will produce a much stronger thesis than a huge matrix with shaky provenance. The repo already warns that one iteration per strategy is not enough and exposes quality checks for taints, placement mismatch, missing recovery data, low **n**, missing provenance, and dirty trees. Use those features ruthlessly. ¹⁸

Ranked experiment portfolio

Experiment	Novelty	Defensibility	Feasibility	Expected signal	Risk	Verdict
Pod-delete layered study	4	5	5	4 at mechanism, 2 at user layer	2	Best fast thesis core
Pod-network-latency or loss on a hot path	4	4	4	5	3	Best positive-result experiment
Redis latency on cart path	5	4	3	5	3	Best ambitious, workload-specific experiment
Node-memory-hog under load	4	3	3	4	4	Strong but confound-prone

Experiment	Novelty	Defensibility	Feasibility	Expected signal	Risk	Verdict
Clean rerun of the load-spike locality result	3	3 now, 4 if cleanly rerun	4	5	3	Worth doing, but do not anchor the thesis on current evidence
Pod-cpu-hog	2	2	5	1 to 2	2	Weak unless you reconfigure limits and load
Disk I/O stress	2	2	4	1 to 2	3	Probably boring for Online Boutique

Those rankings are driven by your repo's current evidence plus official Litmus semantics. Litmus documents pod-delete as forced pod failure and recovery checking; pod-network-latency and pod-network-loss as traffic-control network degradation; pod-cpu-hog as container-level CPU stress; node-memory-hog as node memory exhaustion that can induce unschedulability or eviction; and node/pod IO stress as shared disk stress experiments. For Online Boutique specifically, the cart path's Redis dependency makes Redis/path latency much more likely to generate a route-specific user-visible result than CPU or disk stress on lightly loaded stateless services. ¹⁹

Detailed experiment designs

Pod-delete layered study

Field	Design
Hypothesis	Under single-replica <code>pod-delete</code> , placement changes reconvergence metrics much more than user-visible outcome metrics; aggregate resilience scores do not rank strategies reliably.
Independent variable	Placement strategy: <code>default</code> , <code>colocate</code> , <code>spread</code> , <code>best-fit</code> ; optionally <code>dependency-aware</code> as a fifth strategy.
Dependent variables	Aggregate score; conntack flush %; CoreDNS p99; recovery split (<code>deletionToScheduled</code> , <code>scheduledToReady</code>); dependent-route p95/p99 and error rate; control-route p95/p99 and error rate.
Controls	Same fault target (<code>productcatalogservice</code>), same steady load profile, same cluster version and CNI, same namespace reset between iterations, randomized strategy order within each run block, same replica count.
Expected mechanism	Endpoint removal and reprogramming change kube-proxy / conntack / DNS state; cross-node dependents experience more reconvergence churn than co-located placements.

Field	Design
Metrics to collect	<code>conntrack_entries_per_node</code> , <code>coredns_request_duration_p99</code> , <code>tcp_retransmit_rate_per_node</code> , route-level Locust plus in-pod latency via <code>routeView</code> , recovery watcher events, <code>placementMatchRates</code> , <code>metricAvailability</code> .
Statistical tests	Mixed-effects models or blocked nonparametric comparisons per metric layer; ICC decomposition for score variance; within-run Spearman on mechanism vs dependent/control routes; Holm-adjusted post-hoc contrasts; report effect sizes.
Minimum repetitions	8 clean repetitions per strategy minimum; 10-12 if CI overlap remains large.
Main threats	Single-replica deletion hard-caps availability, so user-visible effects may be structurally muted; missing kube-proxy metrics; cluster drift; scheduler overriding intended placement.
Why interesting	It turns a vague “placement matters” debate into a sharper finding: where placement matters under churn, and where it does not.

This is the experiment your public docs already almost support. The repo’s own H1–H3 story says score variance is dominated by noise, `spread` versus `colocate` shows a strong conntrack flush separation, and mechanism metrics do not predict dependent-route tails once run-level confounds are controlled. That is already thesis material; it just needs clean reruns, enough replication, and tighter language. ²⁰

Pod-network-latency or pod-network-loss on a hot service

Field	Design
Hypothesis	Under injected network latency or loss, placement has a direct user-visible effect : denser/local placements outperform spread on routes that traverse the affected service, while independent control routes move much less.
Independent variable	Placement strategy × fault intensity, for example <code>50/100/200 ms</code> latency or <code>1/5/10%</code> loss.
Dependent variables	Route p95/p99, route error rate, retry/error category counts, TCP retransmits, control-route latency, optionally service-specific throughput.
Controls	Fixed target service, fixed load shape, liveness probes configured not to restart the service due to test latency, same strategy set and iteration ordering policy.
Expected mechanism	Network degradation hits east-west service calls without killing the pod, so locality should become visible at the user layer rather than being swamped by complete unavailability.
Metrics to collect	<code>routeView</code> , Locust failure classes, in-pod latency summary, network bytes/packets, retransmits, route-specific status distributions.

Field	Design
Statistical tests	Two-factor model or aligned blocked comparisons across strategy and intensity; dependent-vs-control route difference-in-differences; Holm-corrected pairwise contrasts.
Minimum repetitions	6–8 per cell.
Main threats	If the target is a single replica, all dependent requests still traverse it, so effect size depends on whether network locality meaningfully differs by strategy; retry behavior can partially mask true latency cost.
Why interesting	This is the cleanest way to demonstrate a positive placement-dependent result without the confound of full pod loss.

Litmus explicitly documents that pod-network-latency degrades the pod network **without necessarily marking the pod unhealthy for kube-proxy**, which is exactly why it is such a good experimental fault here: it isolates communication degradation instead of recovery from disappearance. Pod-network-loss is similarly a direct tc/netem fault. ²¹

Redis latency on the cart path

Field	Design
Hypothesis	Injecting latency on the Redis-backed cart path produces a strong route-specific placement effect: strategies that reduce cross-node <code>frontend ↔ cartservice ↔ Redis</code> hops improve <code>/cart</code> latency and checkout behavior more than they improve unrelated routes.
Independent variable	Placement strategy, especially relative placement of <code>frontend</code> , <code>cartservice</code> , and Redis; latency level on the Redis path.
Dependent variables	<code>/cart</code> p95/p99, checkout latency, cart error rate, redis operation latency, redis ops/s, homepage/product routes as controls.
Controls	Same user workload, same fault duration and location, frozen service versions, no concurrent autoscaling or replica changes.
Expected mechanism	Cartservice explicitly stores and retrieves shopping-cart data in Redis, so path latency should hit a narrow user-visible slice of the application and expose locality more clearly than a global fault would.
Metrics to collect	RouteView for <code>/cart</code> , Redis ops/s and latency, tracer-like timeline if available, control routes, retransmits if the fault is network based.
Statistical tests	Blocked contrasts on <code>/cart</code> and checkout only, plus dependent/control route difference tests.
Minimum repetitions	6–8 per cell.

Field	Design
Main threats	This likely needs a custom or carefully targeted fault, not just a stock scenario; workload mix must contain enough cart traffic.
Why interesting	It is highly falsifiable and produces a mechanism that matches the business route , which examiners like.

This is the best experiment if you want one route to light up while others stay mostly calm. Online Boutique's public architecture makes that path explicit because cartservice uses Redis. ²²

Node-memory-hog under load

Field	Design
Hypothesis	<code>node-memory-hog</code> creates stronger placement-sensitive degradation than <code>pod-cpu-hog</code> because memory pressure can trigger reclaim, OOMs, evictions, and node conditions that spill over to co-located services.
Independent variable	Placement strategy × load profile × whether the hog targets the node hosting the hot path.
Dependent variables	Route p95/p99, error rate, OOMKill counts, restart counts, node <code>MemoryPressure</code> , pod readiness dips, PSI/memory pressure where available.
Controls	Fixed target node, fixed load, fixed node labels, fixed worker memory capacity, no other background workloads.
Expected mechanism	Memory pressure is harder for cgroup limits and CPU requests to “absorb” than mild CPU saturation; co-located placements should suffer more when the pressured node hosts multiple relevant services.
Metrics to collect	Node conditions, <code>oomKillCount</code> , restarts, memory PSI, pod and node memory usage, readiness counts, route-level errors and tails.
Statistical tests	Strategy contrasts within target-node condition; interaction test with load profile.
Minimum repetitions	8 per cell because this fault is noisy and can destabilize the environment.
Main threats	High confounding risk: eviction cascades, node unschedulability, and restart storms can turn the experiment into a cluster-health test rather than a placement study.
Why interesting	It is the best stock Litmus “contention/failure-domain” fault if you want something substantially stronger than <code>pod-cpu-hog</code> .

Official Litmus docs say `node-memory-hog` exhausts node memory and aims to verify resilience when replicas may be evicted because the node becomes not ready or unschedulable. That is exactly why it is more promising than `pod-cpu-hog` for provoking a placement-sensitive result. ²³

The single best quick thesis set

If you need a thesis you can finish quickly, do this and **nothing else**:

Phase	What to run	Why
Core	Clean rerun of <code>pod-delete</code> on <code>productcatalogservice</code> with <code>default</code> , <code>colocate</code> , <code>spread</code> , <code>best-fit</code> and 8–10 repetitions each	Produces the strongest already-supported layered result
Contrast	One clean positive-control experiment: <code>pod-network-latency</code> on the same dependency chain with 6–8 repetitions for <code>colocate</code> vs <code>spread</code> only	Shows that placement can matter strongly at the user layer under the right fault class
Cleanup	Run <code>doctor --strict</code> on every summary, archive all raw outputs, and report placement match rates and provenance in the appendix	Converts a clever tool into defensible science

That overall plan is excellent because it gives you both sides of the story: a strong **negative result** under churn and a strong **positive result** under network degradation. You then argue that the key distinction is **fault class and measurement layer**, not “which strategy is best forever.” The repo already contains the machinery you need for this, including per-route joins, quality checks, provenance capture, and fault-matrix execution. ²⁴

The stronger ambitious set

If you have real extra time, the ambitious version is:

Layer	Experiment set
Churn	Full four- or five-strategy clean rerun of H1–H3 under <code>pod-delete</code>
Network	Strategy × intensity matrix for <code>pod-network-latency</code> or <code>pod-network-loss</code>
Path-specific	Redis latency on cart path
Contention	<code>node-memory-hog</code> under spike load
Optional extension	Multi-replica anti-affinity variant to show that topology begins to matter again once full outage is no longer guaranteed

That version is genuinely strong, but it is also much easier to screw up. If you attempt this, you must prune strategy count, randomize block order, and treat the multi-replica extension as either a short extension chapter or future work unless it is impeccably clean. Kubernetes’ own availability features such as topology spread, anti-affinity, and PDBs become much more relevant once replicas exist, which is why the multi-replica extension is conceptually important but operationally expensive. ²⁵

Falsifiable hypotheses for the top three designs

Hypothesis	Falsifier
H-A: Under single-replica <code>pod-delete</code> , <code>spread</code> produces a larger conntrack flush than <code>colocate</code> , but dependent-route p95/error differences remain statistically small after controlling for run effects.	If <code>spread</code> does not consistently exceed <code>colocate</code> on conntrack flush, or if mechanism metrics strongly predict dependent-route tails within runs, the layered-decoupling claim fails.
H-B: Under injected pod-network latency, <code>colocate</code> and <code>best-fit</code> outperform <code>spread</code> on dependent-route p95/p99, while control routes show much smaller changes.	If dependent and control routes move similarly, or <code>spread</code> performs as well or better, the locality story fails.
H-C: Under <code>node-memory-hog</code> on the node hosting the hot path, dense placements exhibit more readiness loss, OOM/restart activity, and worse user-visible tails than spread placements.	If memory-pressure indicators do not differ by strategy, or user-visible outcomes remain flat, then memory pressure is not the right contention regime for this workload.

Which candidate fault is most interesting

Here is the blunt ranking you asked for:

Fault family	Placement-dependent user-visible signal	Mechanism clarity	Feasibility	Final judgment
Pod-network-latency / loss	High	High	High	Best built-in fault for a positive placement result
Redis latency	Very high	Very high	Medium	Best workload-specific result if you can implement it cleanly
Node-memory-hog	Medium to high	Medium	Medium	Best stock contention fault, but the noisiest
Pod-delete churn	Low at user layer, high at mechanism layer	Very high	High	Best negative-result thesis core
Pod-cpu-hog	Low	Medium	High	Probably weak in your setup
Disk I/O stress	Low	Low to medium	High	Probably not worth thesis time

The reason `pod-cpu-hog` ranks low is not that CPU interference is uninteresting in general. It ranks low because your own repo already documents a plausible workload-specific reason it may be near-no-op here: with the current resource limits, `pod-cpu-hog` can be CFS-capped, and your preliminary load chapter says

CPU hogs were largely absorbed while genuine load contention was not. Litmus’ own docs also describe pod CPU hog as container-level stress, which further supports the concern that it may not produce the node-level collateral you need. Disk stress ranks low because Online Boutique is mostly a networked, memory-resident stateless app; there is simply less reason to expect the cleanest placement effect there.

26

Claims audit and examiner defense

Your present framing has several strong ideas and several dangerous phrases. The ideas are good. Some of the wording is not. The repo’s own bibliography says the thesis “refutes” three literature-derived hypotheses, but that is too aggressive given the actual scope. Kubernetes itself provides availability-oriented placement mechanisms, and your documented cluster and workload are narrow. What you can say safely is that **some placement intuitions from contention-focused literature did not transfer to your tested churn regime**. That is a precise, bounded claim. 27

Keep, weaken, remove, or move

Claim	Keep	Weaken	Remove	Move to future work	Why
Aggregate resilience scores can be misleading	Yes				Strongly supported by your current evidence and by the layered framing. 28
Mechanism metrics and user-visible outcomes must be analyzed separately	Yes				This is the heart of the thesis. 28
Pod placement affects churn reconvergence signatures	Yes				Strong connttrack evidence supports it. 29
Placement does not matter for resilience		Yes			Only safe as “under single-replica pod-delete in this setup, placement did not yield a stable user-visible effect.” 30
The thesis refutes the placement literature		Yes			Say “finds those intuitions inapplicable in this specific fault class and setup.” 31
Co-location is the best strategy			Yes		Current evidence does not justify a universal ranking.

Claim	Keep	Weaken	Remove	Move to future work	Why
Spread is never the safer choice			Yes		This is too broad and invites a fatal examiner attack, especially because Kubernetes explicitly supports spreading for HA. ³²
<code>pod-cpu-hog</code> is inapplicable here	Yes				Reasonable if you show limits/load context and quality checks. ³³
<code>node-memory-hog</code> is stronger than <code>pod-cpu-hog</code>		Yes			Safe as a hypothesis or design rationale, not as a completed result unless rerun. ³⁴
Load contention inverts L1/L2		Yes			Keep only as a preliminary or rerun result; current docs explicitly warn provenance is dirty. ³⁵

Threats to validity and how to defend them

Threat	Why an examiner will care	Defense strategy
Single-replica design	Pod-delete guarantees temporary disappearance of the only instance, which can swamp topology effects.	Agree. Say this is why the thesis is explicitly about single-replica churn and layered measurements, not replica placement for HA. Point to multi-replica anti-affinity as future work. ³⁶
Small virtualized cluster	Four 4 GiB workers on KVM/QEMU may not generalize.	Agree. Claim bounded external validity. Emphasize direction and mechanism, not absolute latency values. ³⁷
Version sensitivity	kube-proxy and conntrack behavior evolve.	Archive exact Kubernetes, CNI, runtime, ChaosProbe, and commit metadata; present this as a measurement study of a specific environment. ³⁸
Placement mismatch	The scheduler may not actually realize intended placement.	Report <code>placementMatchRates</code> and exclude or flag mismatched runs. ³⁹
Run-to-run drift	Iteration noise can dominate.	Use blocked runs, randomize strategy order, capture pre/post snapshots, and model run as a random or blocking effect. The repo already exposes pre/post snapshots and run-level metadata. ⁴⁰

Threat	Why an examiner will care	Defense strategy
Dirty provenance	Untracked files or missing metadata undermine credibility.	Never quote results from runs that fail <code>doctor --strict</code> ; rerun H4 cleanly or demote it. ⁴¹
Metric availability gaps	Missing Prometheus queries can create fake zeros.	Use <code>metricAvailability</code> and explicitly distinguish “not collected” from “collected zero.” ⁴²
Overclaiming causality	Correlation can be confounded by run-level slowness.	Use dependent and control routes plus within-run correlation analyses; present causal language only for manipulated variables, not for every metric relation. ⁴³

Hostile examiner questions and concise answer strategies

Likely question	Best short answer
“Isn’t this just a tooling project?”	“The tool is necessary infrastructure, but the thesis contribution is empirical: a bounded measurement study showing that placement effects under chaos are fault-class- and metric-layer-specific.”
“Why should I care about a negative result?”	“Because it falsifies a common operational simplification: that a single aggregate resilience score or a generic ‘spread is safer’ intuition is enough. In this regime, it is not.”
“Doesn’t Kubernetes already support availability-aware placement?”	“Yes. That is exactly why I avoid claiming Kubernetes ignores availability. My claim is narrower: upstream placement mechanisms do not guarantee a stable user-visible gain under every fault class.” ³²
“Why single replica? Isn’t that unrealistic?”	“It is a deliberately sharp regime for isolating churn reconvergence from replica redundancy. I present multi-replica anti-affinity as the natural extension, not as a completed claim.”
“How do you know contrack is the mechanism?”	“I do not claim it is the only mechanism. I claim it is the most reproducible observed signature, and that upstream Kubernetes issues document the same class of endpoint-change/contrack problems.” ⁴⁴
“Why trust your H4 locality result?”	“I do not ask you to trust it as finalized evidence. The current public documentation marks it preliminary because of dirty provenance and low <code>n</code> ; either I rerun it cleanly or I report it as a pilot.” ³⁵
“Why not just say spread is best for HA?”	“Because upstream Kubernetes docs say spreading can help HA, but my experiments show that for single-replica churn the user-visible outcome is dominated by disappearance, not topology, and for some latency-dominated regimes locality may win.” ⁴⁵

Likely question	Best short answer
"What is your actual novelty compared with MicroRes?"	"MicroRes builds a resilience index from degradation dissemination. My study asks when those layers diverge under specific fault classes and assesses whether aggregate scores obscure placement effects." 46

What evidence each major claim needs

Claim	Required data	Required test	Required figure/table	Likely objection	Defense
Aggregate score does not rank placements under churn	Per-iteration scores by strategy and run block	ICC / variance decomposition; blocked contrasts; effect sizes	Score distribution plot + variance table	"You were underpowered."	Report achieved power and minimum detectable effect; that is itself part of the result.
Spread changes reconvergence mechanism more than colocate	Conntrack, DNS, retransmit, recovery metrics	Blocked contrast or mixed model	Mechanism violin/ECDF plot	"Maybe it is just random cluster noise."	Show sign consistency across blocks and controls.
Mechanism does not propagate to user-visible outcome under churn	Mechanism metrics plus dependent/control route metrics	Within-run correlation and dependent-vs-control comparison	Scatter with control/dependent overlays	"You used the wrong route."	Show robustness to alternative route classifications.
Network faults expose locality at the user layer	Route p95/p99 and error data under network latency/loss	Strategy × intensity comparisons	Heatmap or interaction plot	"The effect is just fault severity."	Show strategy-by-severity interaction and dependent/control separation.
node-memory-hog is a stronger contention fault than pod-cpu-hog	OOMs, pressure, readiness loss, tails under both faults	Direct paired or blocked comparison	Side-by-side fault comparison table	"You changed too many things."	Same load, same strategies, same target node, only fault changes.

Claim	Required data	Required test	Required figure/table	Likely objection	Defense
H4 load-locality claim	Clean rerun from non-dirty commit with enough n	Blocked contrast on p95/p99	Route-tail ranking plot	"Your current result is not trustworthy."	Agree; only elevate after clean rerun.

Thesis blueprint and completion plan

The most defensible abstract is below. It is written to avoid inflated claims while still sounding like systems research.

Defensible abstract

Abstract. Kubernetes offers multiple placement mechanisms and rich observability, yet it remains unclear when pod placement materially affects resilience under chaos and when aggregate resilience scores obscure that effect. This thesis presents **ChaosProbe**, a Kubernetes chaos-evaluation framework that varies pod-placement strategies, injects LitmusChaos faults into the Online Boutique microservice benchmark, collects Prometheus, Kubernetes, Locust, and application-level metrics, and stores structured experiment data for analysis. Using ChaosProbe, I conduct a layered measurement study across aggregate scores, mechanism-level signals, and user-visible outcomes. The central finding is fault-class-specific: under single-replica pod-delete churn, placement reproducibly changes kernel/network reconvergence signatures, but these differences do not yield a stable user-visible advantage and are poorly captured by aggregate resilience scores. In contrast, latency-dominated faults are expected to expose stronger user-visible placement effects. These results show that placement under chaos should not be evaluated with a single score alone; resilience conclusions depend on both the fault class and the measurement layer. The thesis contributes a reproducible experimental framework, a bounded empirical study, and practical guidance for how to evaluate placement-sensitive resilience claims in Kubernetes. 47

Structure of the manuscript

Chapter	What it must do
Introduction	State the problem as a measurement gap: operators change placement and run chaos, but do not know which layers actually move.
Background	Explain Kubernetes scheduling, topology spread, service routing, EndpointSlice/kube-proxy basics, LitmusChaos, Online Boutique. 48
Related work	Separate scheduler literature, locality/QoS schedulers, chaos-engineering frameworks, resilience scoring frameworks, Kubernetes churn/network behavior.
System design	Describe ChaosProbe architecture, placement strategies, metrics pipeline, Neo4j export, and quality checks. 49

Chapter	What it must do
Experimental methodology	Define fault classes, strategies, controls, metrics, route classification, blocking/randomization, statistics, exclusion criteria.
Results	Present one layer at a time: score, mechanism, user-visible outcome. Do not mix them into one chart soup.
Discussion	Explain why churn differs from network/contention faults; connect to Kubernetes mechanisms and literature.
Limitations	Single replica, small cluster, version/CNI dependence, workload choice, preliminary extensions.
Future work	Multi-replica anti-affinity, more realistic contention, service-mesh interaction, cross-cluster replication.
Reproducibility appendix	Environment, commands, commit hashes, seeds, scenario hashes, <code>doctor --strict</code> outputs, data schema.

What is still missing for the project to feel complete

Area	Missing or risky item
Manuscript	A sharply bounded problem statement; a stronger related-work chapter; explicit limitations; a chapter-level results narrative rather than only docs/scripts
Experiment artifacts	Clean reruns with enough <code>n</code> ; blocked randomization; preserved raw archives; one clearly positive-control fault
Result archives	Every quoted claim needs archived <code>summary.json</code> , exported per-iteration tables, and <code>doctor --strict</code> output
Figures	You need thesis-native figures, not just auto-generated internal charts
Tables	You need formal result tables with effect sizes, not only prose findings
Reproducibility package	Commit hashes, environment snapshot, scenario YAML hashes, seed log, load profile definitions, cluster config
Defense materials	A 10-minute story, hostile-question sheet, one-page memorization summary
Code/data hygiene	Confirm no churn analyses accidentally include <code>cpu-hog</code> ; confirm no claims rely on dirty-tree runs

I could not independently verify every local concern you mentioned from the public repo alone, especially about missing pod-delete archives or accidental classification mistakes. What I **could** verify is that the repo already recognizes these classes of problems: it stores run metadata, exposes placement match rates, has metric-availability guards, and ships a `doctor` command that flags dirty provenance, missing metadata, taints, mismatch, and insufficient sample size. That means the thesis should lean hard on those artifact checks. ⁵⁰

Submission checklist

Group	Required items
Must-have before submission	Clean pod-delete rerun with adequate replication; one positive-control experiment; no quoted dirty-tree results; explicit exclusion criteria; provenance appendix; all raw <code>summary.json</code> files archived; strategy match-rate table; final figures and statistics tables; thesis chapter drafts complete
Should-have for a strong defense	Multi-block randomized execution order; route-level control/dependent comparison; a clean rerun of the load-locality pilot or a Redis-latency experiment; one ablation on strategy set; appendix with <code>doctor --strict</code> reports
Nice-to-have if time remains	Multi-replica anti-affinity extension; richer Neo4j graph queries; public artifact packaging script; defense demo video or replayable run
Unsafe claims to remove	"refuted," "proven," "best strategy," "spread is never safer," "generalizes," "causal" unless tied to the manipulated variable and bounded carefully

Minimum viable thesis plan

The smallest academically defensible version is:

Component	Minimum content
Experiments	<code>pod-delete</code> layered study on 4 strategies $\times$ 8-10 repetitions, plus 1 positive-control network-latency contrast on <code>colocate</code> vs <code>spread</code>
Core figures	Score variance plot, conntrack-flush plot, mechanism-vs-outcome scatter, route-tail contrast plot, provenance/quality table
Core tables	Experimental setup, strategy definitions, statistical summary, threats to validity, evidence-per-claim
Core claims	Aggregate score is noisy under churn; mechanism effects can be strong without user-visible separation; fault class determines whether placement matters at the user layer
Scope language	One workload, one cluster, bounded external validity

Stronger ambitious plan

The stronger version adds:

Component	Added value
Redis-latency or cart-path experiment	Clean route-specific positive result

Component	Added value
Node-memory-hog under load	Stronger contention contrast
Multi-replica extension	Real HA relevance
Broader strategy set	<code>dependency-aware</code> and <code>random</code> can become informative side cases rather than curiosities
Public artifact package	Substantially stronger reproducibility story

Final tables and figures to include

Artifact	Caption goal
Figure: Placement strategies and dependency graph	Show what each strategy changes and which services depend on the chaos target
Figure: Aggregate score distributions under pod-delete	Prove that the aggregate score does not rank placements stably
Figure: Contrack flush by strategy	Prove that placement has a strong mechanism-level effect under churn
Figure: Mechanism vs dependent and control route tails	Prove decoupling between mechanism metrics and user-visible outcome
Figure: Representative churn timeline	Show deletion → endpoint change → reconvergence metrics → route errors in one run
Figure: Network-latency route tails by strategy	Show a positive user-visible placement effect in the right fault class
Table: Experimental design and controls	Defend methodological rigor
Table: Statistical results with effect sizes	Defend not just significance but practical magnitude
Table: Threats to validity	Preempt examiner attacks
Table: Reproducibility manifest	Show exact environment and artifacts

Reproducibility requirements

Requirement	What to archive or record
Raw data	All <code>summary.json</code> files, per-iteration exports, Locust CSVs, Litmus ChaosResults, Kubernetes events, cluster snapshots, and any generated statistics CSVs

Requirement	What to archive or record
Scripts	Every analysis script used for a quoted number, plus one top-level <code>make reproduce-thesis</code> or equivalent
Environment details	Kubernetes version, CNI, container runtime, node counts and memory/CPU, ChaosProbe version, Python version, host OS
Cluster configuration	Scheduler settings, topology labels, taints, resource limits/requests, any node selectors or affinity rules
Randomness	Base seed, per-iteration seed, strategy order per block
Scenario integrity	Hashes of all scenario YAMLS and workload manifests
Code integrity	Git commit hashes for ChaosProbe and workload manifests, dirty/clean flag
Reviewer packaging	One archive with raw runs, one archive with processed tables/figures, one manifest file mapping every thesis figure/table to input files and scripts

The good news is that your repo already stores or references part of this story: `summary.json` includes scenario content, run metadata can include commit and environment details, random placement uses tracked seeds, and the tool can export per-iteration data, summarize runs, estimate power, and run data-quality checks. Build the thesis package around those features instead of inventing a second artifact pipeline. ⁵¹

Final narrative and defense summary

The final narrative arc should be simple.

Opening problem. Operators assume placement and chaos results can be summarized by a single resilience ranking, but Kubernetes microservices sit on multiple layers of behavior: scheduling, endpoint propagation, node-local proxy state, service dependencies, and user-visible routes. ⁵²

Literature gap. Scheduler and placement papers optimize locality, utilization, or QoS; chaos frameworks automate experimentation or resilience indexing; upstream Kubernetes issues document churn-pathology mechanisms; but none of these directly establish when placement effects under chaos appear at the mechanism layer, when they propagate to the user, and when aggregate scores hide them. ⁵³

System contribution. ChaosProbe varies placement, injects controlled faults, collects cross-layer metrics, stores structured outputs, and supports provenance-aware statistical analysis. ⁵⁴

Experiment contribution. The thesis studies one workload in one bounded Kubernetes environment across carefully chosen fault classes, using dependent and control routes plus mechanism metrics and artifact-quality checks. ⁵⁵

Key finding. Under single-replica churn, placement can leave a reproducible footprint in reconvergence mechanisms without yielding a stable user-visible advantage, and aggregate scoring can obscure that

distinction. Under latency-dominated faults, placement is much more likely to matter where users can feel it. 56

Limitation. This is not a universal scheduler theorem; it is a bounded empirical study of a specific workload, cluster, and set of faults. Kubernetes versions, CNI, replication factor, and workload design matter. 57

Practical implication. Do not choose placement strategies by one aggregate resilience score. Evaluate them per fault class and per measurement layer, and always separate mechanism signals from user-visible outcomes. 28

One-page defense summary

What problem am I solving?

Kubernetes operators can vary pod placement and run chaos experiments, but existing practice often collapses everything into a single resilience score or a generic assumption such as “spreading is safer.” In microservices, that is too simplistic because failures propagate across multiple layers: control plane, kube-proxy and DNS state, service dependencies, and user-visible latency. 58

What did I build?

I built ChaosProbe, a framework that applies multiple placement strategies to Online Boutique, injects LitmusChaos faults, collects Prometheus, Kubernetes, Locust, and service-level metrics, and stores structured experiment results for statistical analysis and reproducibility checks. 54

What is the research question?

Under which chaos fault classes does pod placement measurably affect mechanism-level behavior and user-visible outcomes, and when do aggregate resilience scores fail to reveal that effect?

What is the main result?

In the strongest completed experiment, single-replica `pod-delete` churn, placement did **not** produce a stable user-visible ranking, and the aggregate score was too noisy to rank strategies reliably. But placement **did** reproducibly change kernel/network reconvergence signatures, especially conntrack-related behavior. So placement mattered at the mechanism layer more than at the user layer. 43

Why is that interesting?

Because it shows that a negative result is hiding inside a positive mechanism result. If you only look at one score, you miss the story. If you only look at mechanism metrics, you can overclaim user impact. The thesis contribution is the layered measurement discipline that separates those cases. 28

How does this fit the literature?

Scheduler papers justify why placement might matter; MicroRes and related resilience frameworks justify cross-layer measurement; upstream Kubernetes documentation and issues justify the churn-reconvergence mechanism. My work combines those threads in a fault-class-specific Kubernetes chaos study. 59

What are my safe claims?

Aggregate scores can be misleading. Placement effects are fault-class-specific. Under single-replica churn, mechanism-level differences need not propagate to stable user-visible differences. A positive placement result likely requires a latency- or contention-dominated fault. 56

What are my unsafe claims?

I did not prove a universally best strategy. I did not refute the entire placement literature. I did not show that spread is never safer. I did not establish broad external validity beyond my workload, cluster, and versions. ⁶⁰

What would I do next?

Run a clean positive-control experiment with pod-network-latency or Redis latency, then extend to multi-replica anti-affinity to test placement under a truly HA-relevant regime. ⁶¹

Open questions and limitations

The two biggest unresolved questions are whether a **clean positive user-visible placement effect** appears under a built-in network fault, and whether the **preliminary load-locality result** survives a clean rerun with enough replication. Your thesis does not fail if you leave those as open questions. It fails only if you overstate them as already settled. ³⁵

¹ ¹⁸ ²⁴ ⁵⁴ [uva-msc-thesis/chaosprobe/docs/how-to/run-experiments.md](https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/how-to/run-experiments.md) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/how-to/run-experiments.md>

² ³ ⁵ ¹⁷ ²⁰ ²⁶ ²⁸ ²⁹ ³⁰ ³³ ³⁵ ⁴¹ ⁴³ ⁴⁴ ⁵⁶ [uva-msc-thesis/chaosprobe/docs/explanation/hypotheses.md](https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/explanation/hypotheses.md) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/explanation/hypotheses.md>

⁴ ²⁵ ³² ⁴⁵ ⁵⁸ ⁶⁰ <https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>
<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

⁶ ⁷ ¹⁴ ⁴⁸ ⁵² <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>
<https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>

⁸ ⁵³ ⁵⁹ <https://research.google.com/pubs/archive/43438.pdf>
<https://research.google.com/pubs/archive/43438.pdf>

⁹ <https://www.carlocentofanti.it/papers/latency-aware-kubernetes.pdf>
<https://www.carlocentofanti.it/papers/latency-aware-kubernetes.pdf>

¹⁰ ²² <https://github.com/googlecloudplatform/microservices-demo>
<https://github.com/googlecloudplatform/microservices-demo>

¹¹ ¹⁵ ⁴⁶ ⁴⁷ <https://2024.issta.org/details/issta-2024-papers/27/MicroRes-Versatile-Resilience-Profiling-in-Microservices-via-Degradation-Disseminati>
<https://2024.issta.org/details/issta-2024-papers/27/MicroRes-Versatile-Resilience-Profiling-in-Microservices-via-Degradation-Disseminati>

¹² ¹⁶ <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
<https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>

¹³ ³⁶ ³⁷ ³⁸ ³⁹ ⁴⁰ ⁴² ⁴⁹ ⁵⁰ ⁵¹ ⁵⁵ ⁵⁷ <https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/TECHNICAL.md>
<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/TECHNICAL.md>

¹⁹ <https://litmuschaos.github.io/litmus/experiments/categories/pods/pod-delete/>
<https://litmuschaos.github.io/litmus/experiments/categories/pods/pod-delete/>

²¹ ⁶¹ <https://litmuschaos.github.io/litmus/experiments/categories/pods/pod-network-latency/>
<https://litmuschaos.github.io/litmus/experiments/categories/pods/pod-network-latency/>

²³ ³⁴ <https://litmuschaos.github.io/litmus/experiments/categories/nodes/node-memory-hog/>
<https://litmuschaos.github.io/litmus/experiments/categories/nodes/node-memory-hog/>

²⁷ ³¹ [uva-msc-thesis/references.md at main · yhu02/uva-msc-thesis · GitHub](https://github.com/yhu02/uva-msc-thesis/blob/main/references.md)
<https://github.com/yhu02/uva-msc-thesis/blob/main/references.md>